Projet Simulation Numérique

But : mettre en œuvre des algorithmes "récents" sur une seule application (image

Liens : Calcul scientifique - Optimisation - Algèbre linéaire

Langage de programmation : Python

Deadline : 11 avril 2025

Thématique : Algorithmes de factorisation structurée pour l'analyse de données.

Soit $A \in \mathbb{R}^{m \times n}$ $(m \geqslant n)$

- Approximation de rang faible :

Soit $k \in \mathbb{N}$ tel que $k \leqslant n$
On cherche à approcher A par une matrice de rang faible $\hat{A} \in \mathbb{R}^{m \times n}$
telle que $\text{rang}(\hat{A}) = k$ et $A = \hat{A} + E$
avec E une matrice d'erreur d'approximation

- Un premier exemple d'algorithme "possible" :

SVD tronquée à l'ordre k

$\hat{A} = U_k \Sigma_k V_k^T$ avec $U_k \in \mathbb{R}^{m \times k}$ (k premières colonnes de U)
$V_k \in \mathbb{R}^{n \times k}$ (k premières colonnes de V)

Critiques :
- Coût opératoire lorsque A est de grande taille
- Perte de la positivité des coefficients dans U_k et/ou V_k
- Perte éventuelle de la structure si A est une matrice creuse

Soit $k \in \mathbb{N}$, on cherche à construire une approximation de rang faible de A de la forme $A = \hat{A} + E$

$$\hat{A} = \underbrace{\begin{bmatrix} \quad \\ \\ \quad \end{bmatrix}}_{I} \underbrace{\begin{bmatrix} \quad \end{bmatrix}}_{D}$$

avec $I \in \mathbb{R}^{m \times k}$ et $D \in \mathbb{R}^{k \times n}$
déduites directement de A

Interpolative decomposition :

Soit $A \in \mathbb{R}^{m \times n}$ $(m \geq n)$ de rang k avec $k \leq n$

Il existe alors $C \in \mathbb{R}^{m \times k}$ et $Z \in \mathbb{R}^{k \times n}$ tels que $A = CZ$

où C est une matrice constituée de k colonnes de A
et Z est une matrice telle que $\max_{ij} |z_{ij}| \leq 1$

En pratique, on cherche une forme $A = CZ + E$

L'idée est de trouver un algorithme tel que

$$\sigma_{k+1}(A) \leq \|E\|_2 \leq \tilde{c}_{m,n} \, \sigma_{k+1}(A)$$

avec $\tilde{c}_{m,m} \in \mathbb{R}$ tel que $\tilde{c}_{m,n} \geq 1$

$\|A - \hat{A}_k\|_2 = \sigma_{k+1}(A)$ (Théorème d'Eckart-Young

Ce que l'on souhaite :
- C préserve les propriétés de A
- algorithme économique en mémoire
- utilisation pour l'analyse de données

Une première possibilité pour construire C et Z
$\rightarrow$ QR avec pivotage de colonnes Rank Revealing QR (RRQR)

$AP_c = QR$
avec P_c matrice de permutation de colonnes ($P_c \in \mathbb{R}^{n \times n}$)

On a avec cette méthode

$$AP_c = [Q_1 \;\; Q_2] \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix}$$ avec $R_{11} \in \mathbb{R}^{k \times k}$ inversible et $Q_1 \in \mathbb{R}^{m \times k}$

Avec cette méthode, comme approximation de A, on choisit $\hat{A} \in \mathbb{R}^{m \times n}$ sous la forme

$$\hat{A} = Q_1 [R_{11} \;\; R_{12}] P_c^T = \underbrace{Q_1 R_{11}}_{\substack{C \\ \in \mathbb{R}^{m \times k}}} \underbrace{[I_k \;\; R_{11}^{-1} R_{12}] P_c^T}_{\substack{Z \\ \in \mathbb{R}^{k \times n}}}$$ Column ID

$T = R_{11}^{-1} R_{12}$ "T. expansion matrix"

$R_{11} T = R_{12}$: Résolution d'un système structuré à multiples seconds membres

$E = A - \hat{A}$

Après un calcul, on a $E = [0 \quad Q_2 R_{22}] P_c^T$

$\|E\|_2 = \|R_{22}\|_2$

Row ID:

$\hat{A} = XR$ avec $X \in \mathbb{R}^{m \times k}$ et $R \in \mathbb{R}^{k \times n}$

et R composée de k lignes de A

Double ID:

$\hat{A} = X A_s Z$ avec $X \in \mathbb{R}^{m \times k}$, $Z \in \mathbb{R}^{k \times n}$ et $A_s \in \mathbb{R}^{k \times k}$ sous-bloc de A

Est-il possible de s'affranchir de la méthode QR avec pivotage de colonnes ?

Le second volet du projet consistera à construire $\hat{A}$ avec des algorithmes basés sur l'aléatoire

1 - Algorithmes déterministe
2 - Algorithmes aléatoires
3 - Analyse de performances
4 -